

The Ultimate Web Development Handbook



Operators in JavaScript

Operators are symbols used to **perform operations** on values and variables. For example, you use `+` to add two numbers, `=` to assign values, and `==` to compare values.

1. Arithmetic Operators

Used to perform mathematical operations.

Operator	Description	Example	Result
<code>+</code>	Addition	<code>5 + 2</code>	<code>7</code>
<code>-</code>	Subtraction	<code>5 - 2</code>	<code>3</code>
<code>*</code>	Multiplication	<code>5 * 2</code>	<code>10</code>
<code>/</code>	Division	<code>10 / 2</code>	<code>5</code>
<code>%</code>	Modulus (Remainder)	<code>5 % 2</code>	<code>1</code>
<code>**</code>	Exponentiation	<code>2 ** 3</code>	<code>8</code>
<code>++</code>	Increment	<code>let x = 1; x++</code>	<code>2</code>
<code>--</code>	Decrement	<code>let x = 2; x--</code>	<code>1</code>

2. Assignment Operators

Used to assign values to variables.

Operator	Example	Same As
<code>=</code>	<code>x = 5</code>	Assign 5 to x
<code>+=</code>	<code>x += 2</code>	<code>x = x + 2</code>
<code>-=</code>	<code>x -= 3</code>	<code>x = x - 3</code>
<code>*=</code>	<code>x *= 4</code>	<code>x = x * 4</code>
<code>/=</code>	<code>x /= 2</code>	<code>x = x / 2</code>
<code>%=</code>	<code>x %= 2</code>	<code>x = x % 2</code>

3. Comparison Operators

Used to compare two values. They return a Boolean (`true` or `false`).

Operator	Description	Example	Result
<code>==</code>	Equal (loose)	<code>5 == '5'</code>	<code>true</code>
<code>===</code>	Strict equal (type + value)	<code>5 === '5'</code>	<code>false</code>
<code>!=</code>	Not equal	<code>5 != '5'</code>	<code>false</code>
<code>!==</code>	Strict not equal	<code>5 !== '5'</code>	<code>true</code>
<code>></code>	Greater than	<code>5 > 3</code>	<code>true</code>
<code><</code>	Less than	<code>5 < 3</code>	<code>false</code>
<code>>=</code>	Greater than or equal	<code>5 >= 5</code>	<code>true</code>
<code><=</code>	Less than or equal	<code>3 <= 5</code>	<code>true</code>

4. Logical Operators

Used to combine multiple conditions.

Operator	Description	Example	Result
&&	Logical AND	true && false	false
	Logical OR	true false	true
!	Logical NOT	!true	false

Example:

```
let age = 20;
if (age > 18 && age < 60) {
  console.log("You are eligible");
}
```

5. Ternary Operator

A shorthand way to write simple `if...else`.

```
let age = 17;
let result = age >= 18 ? "Adult" : "Minor";
console.log(result); // "Minor"
```

Summary

- Use arithmetic operators to perform math.
- Use assignment operators to update variables.
- Use comparison and logical operators for decision making.
- Ternary operator is a cleaner way to write simple conditions.

If-Else Statements in JavaScript

What is an If-Else Statement?

An `if` statement is used to run a block of code only if a specified condition is true. You can use `else` or `else if` to run different blocks of code based on different conditions.

Basic Syntax

```
if (condition) {  
    // code to run if condition is true  
} else {  
    // code to run if condition is false  
}
```

Example

```
let age = 18;  
  
if (age >= 18) {  
    console.log("You are an adult.");  
} else {  
    console.log("You are a minor.");  
}
```

if-else-if Ladder

You can check multiple conditions using `else if`:

```
let score = 85;

if (score >= 90) {
  console.log("Grade: A");
} else if (score >= 80) {
  console.log("Grade: B");
} else if (score >= 70) {
  console.log("Grade: C");
} else {
  console.log("Grade: F");
}
```

Nested if Statements

You can place one `if` statement inside another:

```
let age = 25;
let hasID = true;

if (age >= 18) {
  if (hasID) {
    console.log("Access granted.");
  } else {
    console.log("ID required.");
  }
} else {
  console.log("Access denied. You must be at least 18.");
}
```

Using the Ternary Operator (Short Form)

The ternary operator is a shorter way to write simple `if-else` statements:

```
let isLoggedIn = true;

let message = isLoggedIn ? "Welcome back!" : "Please log in.";
console.log(message);
```

Summary

- Use `if` to test a condition.
- Use `else` for an alternative block if the condition is false.
- Use `else if` to test multiple conditions.
- The ternary operator is a compact way to write simple `if-else` logic.